



GUÍA DE INTEGRACIONES DE IA - Audio a Texto



ABACUS AI + GROQ - INTEGRACIÓN COMPLETA



1. ABACUS AI INTEGRATION

A. Configuración Inicial

```
// lib/abacus-ai.js
class AbacusAIService {
  constructor() {
    this.apiKey = process.env.ABACUS_AI_API_KEY;
    this.baseUrl = process.env.ABACUS_AI_BASE_URL || 'https://api.abacus.ai';
  }

  async transcribeAudio(audioBuffer, options = {}) {
    const formData = new FormData();
    formData.append('audio', new Blob([audioBuffer]), 'audio.wav');
    formData.append('model', options.model || 'whisper-large-v3');
    formData.append('language', options.language || 'es');
    formData.append('response_format', 'json');
    formData.append('temperature', options.temperature || 0);

    const response = await fetch(`${this.baseUrl}/v1/audio/transcriptions`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${this.apiKey}`,
      },
      body: formData
    });

    if (!response.ok) {
      throw new Error(`Abacus AI Error: ${response.statusText}`);
    }

    const result = await response.json();
    return {
      text: result.text,
      confidence: result.confidence || 1.0,
      segments: result.segments || [],
      language: result.language,
      provider: 'abacus-ai',
      model: options.model,
      duration: result.duration
    };
  }

  async getAvailableModels() {
    const response = await fetch(`${this.baseUrl}/v1/models`, {
      headers: {
        'Authorization': `Bearer ${this.apiKey}`,
      }
    });

    const models = await response.json();
    return models.data.filter(model =>
      model.id.includes('whisper') ||
      model.id.includes('wav2vec') ||
      model.id.includes('speech')
    );
  }
}
```

B. Modelos Abacus AI Disponibles

```
const ABACUS_MODELS = {
  'whisper-large-v3': {
    name: 'Whisper Large V3',
    description: 'Máxima precisión, ideal para producción',
    costPerMinute: 0.05,
    maxFileSize: '25MB',
    languages: ['es', 'en', 'pt', 'fr', 'de'],
    accuracy: '95%+'
  },
  'whisper-medium': {
    name: 'Whisper Medium',
    description: 'Balance velocidad/precisión',
    costPerMinute: 0.03,
    maxFileSize: '25MB',
    languages: ['es', 'en', 'pt', 'fr'],
    accuracy: '92%+'
  },
  'wav2vec2-large-xlsr': {
    name: 'Wav2Vec2 XLSR',
    description: 'Optimizado para múltiples idiomas',
    costPerMinute: 0.02,
    maxFileSize: '25MB',
    languages: ['es', 'en', 'pt'],
    accuracy: '89%+'
  },
  'custom-chile-model': {
    name: 'Modelo Chile Personalizado',
    description: 'Entrenado para español chileno',
    costPerMinute: 0.04,
    maxFileSize: '25MB',
    languages: ['es-CL'],
    accuracy: '97%+ (Chile)'
  }
};
```

⚡ 2. GROQ API INTEGRATION

A. Configuración Groq

```
// lib/groq-ai.js
import Groq from 'groq-sdk';

class GroqService {
  constructor() {
    this.client = new Groq({
      apiKey: process.env.GROQ_API_KEY,
    });
  }

  async transcribeAudio(audioFile, options = {}) {
    try {
      const transcription = await this.client.audio.transcriptions.create({
        file: audioFile, // File object
        model: options.model || 'whisper-large-v3',
        language: options.language || 'es',
        response_format: options.format || 'verbose_json',
        temperature: options.temperature || 0,
        prompt: options.prompt || "Transcripción de audio publicitario en español chileno"
      });

      return {
        text: transcription.text,
        confidence: this.calculateConfidence(transcription),
        segments: transcription.segments || [],
        language: transcription.language,
        provider: 'groq',
        model: options.model,
        duration: transcription.duration,
        words: transcription.words || []
      };
    } catch (error) {
      throw new Error(`Groq API Error: ${error.message}`);
    }
  }

  calculateConfidence(transcription) {
    // Calcular confidence basado en segments
    if (!transcription.segments) return 1.0;

    const avgConfidence = transcription.segments.reduce((sum, segment) => {
      return sum + (segment.avg_logprob || 0);
    }, 0) / transcription.segments.length;

    // Convertir logprob a percentage
    return Math.max(0, Math.min(1, Math.exp(avgConfidence)));
  }

  async getAvailableModels() {
    const models = await this.client.models.list();
    return models.data.filter(model =>
      model.id.includes('whisper')
    );
  }
}
```

B. Modelos Groq Disponibles

```
const GROQ_MODELS = {
  'whisper-large-v3': {
    name: 'Whisper Large V3 (Groq)',
    description: 'Ultra-rápido con Groq chips',
    costPerMinute: 0.01,
    maxFileSize: '25MB',
    languages: ['es', 'en', 'pt', 'fr', 'de', 'it'],
    accuracy: '95%',
    speed: 'Ultra-rápido (10x más rápido)',
    contextLength: 30
  },
  'distil-whisper-large-v3': {
    name: 'Distil-Whisper Large V3',
    description: 'Optimizado para velocidad extrema',
    costPerMinute: 0.008,
    maxFileSize: '25MB',
    languages: ['es', 'en'],
    accuracy: '92%',
    speed: 'Extremadamente rápido (15x más rápido)',
    contextLength: 30
  }
};
```

3. SERVICIO UNIFICADO DE TRANSCRIPCIÓN

A. Balanceador de Carga y Fallback

```
// lib/transcription-service.js
class TranscriptionService {
  constructor() {
    this.abacus = new AbacusAIService();
    this.groq = new GroqService();
    this.providers = ['groq', 'abacus']; // Groq primero por velocidad
    this.maxRetries = 3;
  }

  async transcribe(audioBuffer, options = {}) {
    const provider = options.provider || 'auto';

    if (provider === 'auto') {
      return this.transcribeWithFallback(audioBuffer, options);
    }

    return this.transcribeWithProvider(audioBuffer, provider, options);
  }

  async transcribeWithFallback(audioBuffer, options) {
    for (const providerName of this.providers) {
      try {
        console.log(`🔊 Intentando transcripción con ${providerName}...`);
        const result = await this.transcribeWithProvider(audioBuffer, providerName, options);
        console.log(`✅ Transcripción exitosa con ${providerName}`);
        return result;
      } catch (error) {
        console.error(`❌ Error con ${providerName}:`, error.message);
        continue;
      }
    }

    throw new Error('Todos los proveedores de transcripción fallaron');
  }

  async transcribeWithProvider(audioBuffer, provider, options) {
    const startTime = Date.now();

    let result;
    switch (provider) {
      case 'groq':
        result = await this.groq.transcribeAudio(audioBuffer, options);
        break;
      case 'abacus':
        result = await this.abacus.transcribeAudio(audioBuffer, options);
        break;
      default:
        throw new Error(`Proveedor desconocido: ${provider}`);
    }

    const processingTime = Date.now() - startTime;

    return {
      ...result,
      processingTimeMs: processingTime,
      timestamp: new Date().toISOString()
    };
  }

  async getBestProvider(audioLength, priority = 'speed') {
    // Lógica para seleccionar el mejor proveedor
  }
}
```

```

    if (priority === 'speed' && audioLength < 300) { // < 5 minutos
      return 'groq'; // Groq es más rápido
    }

    if (priority === 'accuracy') {
      return 'abacus'; // Abacus puede tener mejores modelos custom
    }

    if (priority === 'cost') {
      return 'groq'; // Groq generalmente más barato
    }

    return 'groq'; // Default
  }

  async getProviderStatus() {
    const status = {
      groq: { available: false, latency: null },
      abacus: { available: false, latency: null }
    };

    // Test Groq
    try {
      const start = Date.now();
      await this.groq.getAvailableModels();
      status.groq = {
        available: true,
        latency: Date.now() - start
      };
    } catch (error) {
      console.error('Groq health check failed:', error);
    }

    // Test Abacus
    try {
      const start = Date.now();
      await this.abacus.getAvailableModels();
      status.abacus = {
        available: true,
        latency: Date.now() - start
      };
    } catch (error) {
      console.error('Abacus health check failed:', error);
    }

    return status;
  }
}

export default TranscriptionService;

```



4. MONITOREO Y ANALYTICS

A. Métricas de Performance

```
// lib/transcription-analytics.js
class TranscriptionAnalytics {
  constructor() {
    this.metrics = {
      totalTranscriptions: 0,
      successRate: {},
      avgProcessingTime: {},
      costTracking: {},
      accuracyScores: {}
    };
  }

  trackTranscription(result, provider, cost, audioLength) {
    this.metrics.totalTranscriptions++;

    // Success rate
    if (!this.metrics.successRate[provider]) {
      this.metrics.successRate[provider] = { success: 0, total: 0 };
    }
    this.metrics.successRate[provider].total++;
    this.metrics.successRate[provider].success++;

    // Processing time
    if (!this.metrics.avgProcessingTime[provider]) {
      this.metrics.avgProcessingTime[provider] = [];
    }
    this.metrics.avgProcessingTime[provider].push(result.processingTimeMs);

    // Cost tracking
    if (!this.metrics.costTracking[provider]) {
      this.metrics.costTracking[provider] = 0;
    }
    this.metrics.costTracking[provider] += cost;

    // Accuracy
    if (!this.metrics.accuracyScores[provider]) {
      this.metrics.accuracyScores[provider] = [];
    }
    this.metrics.accuracyScores[provider].push(result.confidence);
  }

  getOptimalProvider(audioLength, budget) {
    // Algoritmo para recomendar el mejor proveedor
    const providers = ['groq', 'abacus'];
    let bestProvider = 'groq';
    let bestScore = 0;

    providers.forEach(provider => {
      const successRate = this.getSuccessRate(provider);
      const avgTime = this.getAvgProcessingTime(provider);
      const avgAccuracy = this.getAvgAccuracy(provider);
      const costPer = this.getCostPerMinute(provider);

      // Score weighted: accuracy(40%) + speed(30%) + success(20%) + cost(10%)
      const score = (
        avgAccuracy * 0.4 +
        (1 / avgTime) * 0.3 + // Inverso del tiempo
        successRate * 0.2 +
        (1 / costPer) * 0.1 // Inverso del costo
      );

      if (score > bestScore) {

```

```
        bestScore = score;
        bestProvider = provider;
    }
});

return {
    provider: bestProvider,
    score: bestScore,
    reasoning: this.getProviderReasoning(bestProvider)
};
}
```

5. API ENDPOINTS NECESARIOS

A. API de Transcripción

```
// pages/api/transcription/process.js
import TranscriptionService from '@lib/transcription-service';
import { authenticate } from '@lib/auth';

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  try {
    const user = await authenticate(req);
    if (!user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }

    const { audioFile, options = {} } = req.body;

    const transcriptionService = new TranscriptionService();
    const result = await transcriptionService.transcribe(audioFile, {
      provider: options.provider || 'auto',
      model: options.model,
      language: 'es',
      temperature: 0,
      ...options
    });

    // Guardar en base de datos
    await saveTranscriptionResult(result, user.id);

    res.status(200).json({
      success: true,
      transcription: result
    });
  } catch (error) {
    console.error('Transcription API Error:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
}
```

B. API de Estado de Proveedores

```
// pages/api/transcription/providers/status.js
export default async function handler(req, res) {
  try {
    const transcriptionService = new TranscriptionService();
    const status = await transcriptionService.getProviderStatus();

    res.status(200).json({
      success: true,
      providers: status,
      recommendation: await transcriptionService.getBestProvider(300, 'balanced')
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
}
```

⚙️ 6. CONFIGURACIÓN VARIABLES DE ENTORNO

```
# .env.local

# === ABACUS AI ===
ABACUS_AI_API_KEY=your_abacus_api_key_here
ABACUS_AI_BASE_URL=https://api.abacus.ai
ABACUS_AI_MODEL_DEFAULT=whisper-large-v3

# === GROQ API ===
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL_DEFAULT=whisper-large-v3

# === TRANSCRIPTION SETTINGS ===
TRANSCRIPTION_DEFAULT_PROVIDER=auto
TRANSCRIPTION_MAX_RETRIES=3
TRANSCRIPTION_TIMEOUT=300000
TRANSCRIPTION_CHUNK_SIZE=25MB

# === COST CONTROL ===
MAX_TRANSCRIPTION_COST_PER_HOUR=50.00
COST_ALERT_THRESHOLD=80.00
DAILY_TRANSCRIPTION_LIMIT=1000

# === PERFORMANCE ===
TRANSCRIPTION_CACHE_TTL=3600
ENABLE_TRANSCRIPTION_CACHE=true
MAX_CONCURRENT_TRANSCRIPTIONS=5
```

7. TESTING DE LAS INTEGRACIONES

```
// tests/transcription.test.js
describe('Transcription Services', () => {
  test('Abacus AI transcription works', async () => {
    const service = new AbacusAIService();
    const audioBuffer = fs.readFileSync('test-audio.wav');

    const result = await service.transcribeAudio(audioBuffer, {
      model: 'whisper-large-v3',
      language: 'es'
    });

    expect(result.text).toBeDefined();
    expect(result.confidence).toBeGreaterThan(0.8);
    expect(result.provider).toBe('abacus-ai');
  });

  test('Groq transcription works', async () => {
    const service = new GroqService();
    const audioFile = new File([audioBuffer], 'test.wav');

    const result = await service.transcribeAudio(audioFile);

    expect(result.text).toBeDefined();
    expect(result.provider).toBe('groq');
  });

  test('Fallback system works', async () => {
    const service = new TranscriptionService();
    // Mock Groq to fail
    service.groq.transcribeAudio = jest.fn().mockRejectedValue(new Error('API Error'));

    const result = await service.transcribe(audioBuffer);
    expect(result.provider).toBe('abacus-ai');
  });
});
```

NEXT STEPS PARA IMPLEMENTAR

Paso 1: Obtener API Keys

1. **Abacus AI:** Crear cuenta en abacus.ai (<https://abacus.ai>)
2. **Groq:** Registro en console.groq.com (<https://console.groq.com>)

Paso 2: Implementar Servicios

1. Crear los archivos de servicios mostrados arriba
2. Configurar variables de entorno
3. Instalar dependencias: `npm install groq-sdk form-data`

Paso 3: Testing

1. Probar con audio de muestra
2. Verificar ambos proveedores funcionan
3. Probar sistema de fallback

Paso 4: Integración Completa

1. Conectar con sistema de captura de audio
2. Implementar detección de frases post-transcripción
3. Guardar resultados en Supabase

¡Con estas integraciones tendrás transcripción de audio robusta, rápida y con fallback automático! 🎤

